

Curso: Ciência Da Computação

Disciplina: Desenvolvimento de Sistemas

Professor: Felipe Pires

Grupo: Isabella Sena, Luísa Souza e Maria Eduarda Campelo

SISTEMA DE GESTÃO COMERCIAL (SGC)

Documentação do Sistema Swing

1 INTRODUÇÃO	3
2 OBJETIVOS DO SISTEMA	3
3 TECNOLOGIAS UTILIZADAS	3
4 ARQUITETURA DO SISTEMA	4
5 ESTRUTURA DE PACOTES	5
5.1 Pacote dto	5
5.1.1 Classe ClienteDTO	5
5.1.2 Classe ClienterequestDTO	6
5.1.3 Classe ProdutoDTO	6
5.1.4 Classe ProdutoRequestDTO	6
5.1.5 Classe LoginRequestDTO	6
5.1.6 Classe LoginResponseDTO	7
5.1.7 Classe ItemVendaRequestDTO	7
5.1.8 Classe ItemVendaResponseDTO	7
5.1.9 Classe VendaRequestDTO	7
5.1.10 Classe RelatorioVendaDTO	7
5.2 Pacote service	7
5.2.1 Camada De Serviços	7
5.3 Pacote telas	8
5.3.1 TelaLogin	8
5.3.2 TelaPrincipal	9
5.3.3 TelaClientes	9
5.3.4 TelaProdutos	9
5.3.5 TelaVendas	10
5.5.6 TelaRelatorios	10
6 FLUXOS OPERACIONAIS	11
7 BIBLIOTECAS UTILIZADAS	12
8 CONCLUSÃO	12

1 INTRODUÇÃO

O Sistema de Gerenciamento Comercial (SGC) consiste em uma aplicação desktop desenvolvida utilizando a linguagem Java e a biblioteca Swing para construção da interface gráfica. O sistema foi projetado para realizar o gerenciamento de clientes, produtos, vendas e relatórios comerciais, utilizando uma arquitetura cliente-servidor integrada a uma API REST desenvolvida com Spring Boot.

A aplicação adota arquitetura em camadas e comunicação via protocolo HTTP, permitindo a separação adequada das responsabilidades entre interface gráfica, transporte de dados e regras de negócio.

Além disso, o sistema implementa autenticação baseada em JSON Web Token (JWT), garantindo que apenas usuários devidamente autenticados tenham acesso às funcionalidades disponibilizadas.

2 OBJETIVOS DO SISTEMA

O objetivo principal do Sistema de Gerenciamento Comercial é fornecer uma solução para controle e administração de informações comerciais para a loja de produtos, permitindo o gerenciamento centralizado de clientes, produtos e vendas.

Entre os objetivos específicos destacam-se:

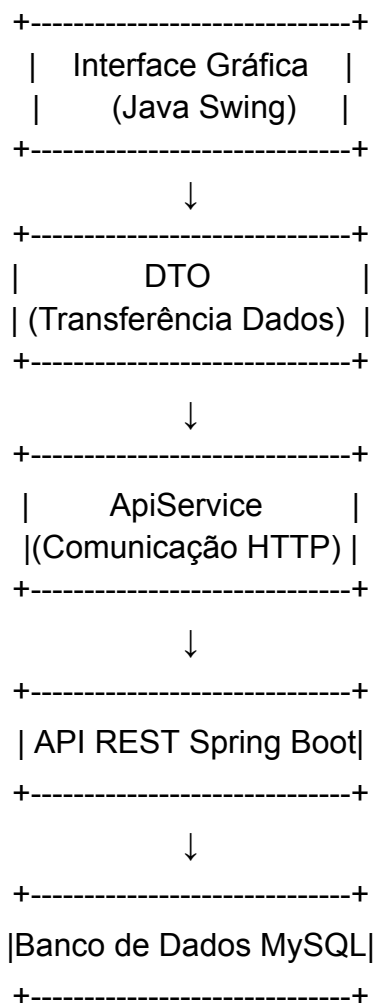
- Realizar autenticação segura de usuários;
- Permitir cadastro, consulta, edição e exclusão de clientes;
- Permitir cadastro, consulta, edição e exclusão de produtos;
- Registrar operações de venda;
- Gerar relatórios de vendas por cliente;
- Integrar uma interface desktop a uma API REST;
- Demonstrar a aplicação prática dos conceitos de Programação Orientada a Objetos.

3 TECNOLOGIAS UTILIZADAS

Tecnologia	Finalidade
Java	Linguagem principal do sistema
Spring Boot	Desenvolvimento da API REST
JWT	Autenticação e autorização
Gson	Conversão de objetos java para JSON
HttpClient	Comunicação HTTP
Maven	Gerenciamento de dependências
MySQL	Persistências de dados

4 ARQUITETURA DO SISTEMA

A arquitetura do sistema foi organizada em múltiplas camadas, visando facilitar a manutenção, reutilização e escalabilidade da aplicação.



Cada camada possui responsabilidades específicas e independentes, reduzindo o acoplamento entre os componentes do sistema.

5 ESTRUTURA DE PACOTES

O front-end Swing encontra-se organizado da seguinte forma:

```
br.com.sgc.swing
|
|--- dto
|--- service
|--- telas
```

5.1 Pacote dto

Contém as classes responsáveis pela transferência de dados entre a aplicação Swing e a API REST.

5.1.1 Classe *ClienteDTO*

- Finalidade: Representar um cliente retornado pela API durante operações de consulta.
- Responsabilidade: Armazenar os dados completos de um cliente retornados pela API.
- Utilização:
 - Listagem de clientes
 - Preenchimento de tabelas
 - Exibição de informações na interface
- Estrutura de Dados

Atributo	Tipo	Descrição
id	Long	Identificador único
nome	String	Nome completo
cpf	String	CPF do cliente
telefone	String	Telefone de contato
e-mail	String	Endereço eletrônico

5.1.2 Classe *ClienterequestDTO*

- Finalidade: Representar os dados enviados ao backend durante operações de cadastro e atualização.
- Estrutura de Dados:
 - Atributo: nome, cpf, telefone, e-mail, endereço.
- Não possui o atributo ID, pois o identificador é gerado automaticamente pelo banco de dados.

5.1.3 Classe *ProdutoDTO*

- Finalidade: Representar um produto retornado pela API.
- Estrutura:

Atributo	Tipo
id	Long
nome	String
descricao	String
preco	BigDecimal
estoque	Integer
estoqueMinimo	Integer

- O tipo BigDecimal foi utilizado para evitar problemas de precisão em operações monetárias.

5.1.4 Classe *ProdutoRequestDTO*

- Finalidade: Representar os dados enviados para cadastro e atualização de produtos
- Estrutura:
 - Campo: nome, descrição, preco, estoque e estoqueMinimo.

5.1.5 Classe *LoginRequestDTO*

- Finalidade: Representar a requisição de autenticação enviada ao backend.
- Campos: username e senha
- Exemplo JSON:

```
{
  "username": "admin",
  "senha": "123456"
}
```

5.1.6 Classe *LoginResponseDTO*

- Finalidade: Representar a resposta da autenticação.
- Estrutura:

Campo	Descrição
username	Usuário autenticado
token	Token JWT
perfil	Perfil de acesso

5.1.7 Classe *ItemVendaRequestDTO*

- Finalidade: Representar um item de venda enviado para registro.
- Campos: produtoId, quantidade

5.1.8 Classe *ItemVendaResponseDTO*

- Finalidade: Representa um item retornado pela API em consultas de vendas.
- Campos: id, nomeProduto, quantidade, preco, subtotal
- Regras de Negócio: Subtotal = Quantidade x Preço Unitário

5.1.9 Classe *VendaRequestDTO*

- Finalidade: Representar uma venda enviada ao backend.
- Campos: clienteId e itens.

5.1.10 Classe *RelatorioVendaDTO*

- Finalidade: Representar os dados consolidados de vendas utilizados nos relatórios.
- Campos: id, data, nomeCliente, usernameUsuario, valorTotal e itens

5.2 Pacote service

Contém a classe responsável pela comunicação HTTP com o backend.

5.2.1 Camada De Serviços

- Classe ApiService
- Responsabilidade Geral: A classe ApiService atua como camada de integração entre o frontend Swing e a API REST. Ela centraliza todas as operações de comunicação HTTP, evitando que as telas possuam responsabilidade direta sobre chamadas externas.

- Principais Responsabilidades:
 - Autenticação de usuários;
 - Consulta de clientes;
 - Cadastro de clientes;
 - Atualização de clientes;
 - Exclusão de clientes;
 - Consulta de produtos;
 - Cadastro de produtos;
 - Atualização de produtos;
 - Exclusão de produtos;
 - Registro de vendas;
 - Consulta de relatórios.
- Benefícios da Centralização:
 - Reutilização de código;
 - Facilidade de manutenção;
 - Redução de duplicidade;
 - Maior organização arquitetural.

5.3 Pacote telas

Contém todas as interfaces gráficas disponíveis ao usuário.

5.3.1 TelaLogin

- Objetivo: Permitir a autenticação do usuário no sistema.

Componente	Tipo
txtUsuario	JTextField
txtSenha	JPasswordField
btnEntrar	JButton

- Fluxo de Funcionamento:

Usuário informa credenciais

↓

ApiService.login()

↓

API valida usuário

↓

JWT retornado

↓

TelaPrincipal

5.3.2 TelaPrincipal

- Objetivo: Atuar como menu principal da aplicação.
- Funcionalidades Disponíveis:
 - Clientes
 - Produtos
 - Vendas
 - Relatórios
 - Encerrar Sessão.
- Particularidade: A tela recebe o token JWT gerado durante o login e o repassa para as demais telas.

5.3.3 TelaClientes

- Objetivo: Realizar o gerenciamento completo de clientes.
- Operações Disponíveis:
 - Cadastro
 - Consulta
 - Atualização
 - Exclusão
- Componentes:

Campo	Variável
Nome	txtNome
CPF	txtCpf
Telefone	txtTelefone
Endereço	txtEndereco

- Tabela: A JTable tblClientes exibe todos os clientes retornados pela API.
- Método carregarClientes() é responsável por:
 - a. Consultar clientes na API;
 - b. Converter a resposta para objetos ClienteDTO;
 - c. Popular a JTable;
 - d. Atualizar a interface.

5.3.4 TelaProdutos

- Objetivo: Gerenciar o cadastro de produtos
- Funcionalidades:
 - Inclusão
 - Consulta

- Atualização
 - Exclusão
- Campos:
 - Nome
 - Descrição
 - Preço
 - Estoque
 - Estoque Mínimo
- Método `carregarProdutos()`: Responsável pela atualização dinâmica da tabela de produtos.

5.3.5 TelaVendas

- Objetivo: Registrar vendas realizadas para clientes cadastrados.
- Dados Necessários:
 - Cliente
 - Produto
 - Quantidade
- Processo:

TelaVendas

↓

VendaRequestDTO

↓

ItemVendaRequestDTO

↓

ApiService

↓

Backend

↓

Banco de Dados

5.5.6 TelaRelatorios

- Objetivo: Consultar vendas realizadas por cliente.
- Funcionalidades:
 - Pesquisa por cliente;
 - Consulta de histórico de vendas;
 - Visualização de valores totais.
- Informações Exibidas:
 - Identificador da venda;
 - Nome do cliente;
 - Usuário responsável;
 - Data da venda;

- Valor total.

6 FLUXOS OPERACIONAIS

- Fluxo de Login

TelaLogin



LoginRequestDTO



ApiService



POST /auth/login



LoginResponseDTO



TelaPrincipal

- Fluxo de Cadastro de Cliente

TelaClientes



ClienteRequestDTO



ApiService



POST /clientes



Backend



Banco de Dados

- Fluxo de Cadastro de Produto

TelaProdutos



ProdutoRequestDTO



ApiService



POST /produtos



Banco de Dados

- Fluxo de Registro de Venda

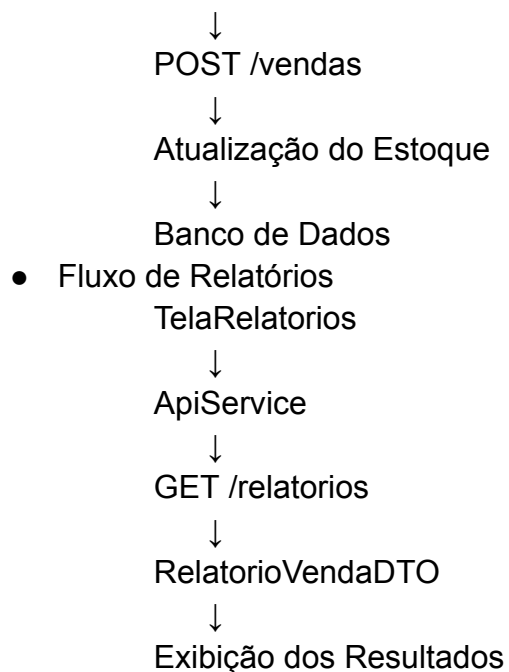
TelaVendas



VendaRequestDTO



ApiService



7 BIBLIOTECAS UTILIZADAS

Biblioteca	Função
Java Swing	Interface gráfica
HttpClient	Comunicação HTTP
Gson	Conversão JSON
JTable	Exibição de dados
EventQueue	Gerenciamento de janelas

8 CONCLUSÃO

O Sistema de Gerenciamento Comercial demonstra a aplicação integrada de conceitos fundamentais de desenvolvimento de software, arquitetura em camadas, comunicação cliente-servidor, consumo de APIs REST e autenticação baseada em JWT. A separação entre interface gráfica, objetos de transferência de dados e camada de serviços contribui para uma arquitetura mais organizada, modular e de fácil manutenção. Além disso, a utilização do Java Swing permitiu a construção de uma interface desktop funcional e integrada ao backend Spring Boot, proporcionando uma solução completa para gerenciamento comercial.

